

# ส่งต่อให้ทีม Power Platform Skill Suite

ชุด Claude Skills ที่พาเราจาก requirement → แบบที่ user เห็นชอบแล้ว → Power Apps + SharePoint + Power Automate ที่ใช้งานได้จริง ด้วย prompt สั้น ๆ และ token เท่าที่ Pro plan รับไหว

ผมเอากฎของ Power Apps ทั้งหมดที่เราเก็บตัวมาแล้ว ไปใส่ไว้ให้ Claude รู้ ตั้งแต่ก่อนพิมพ์ **code** บรรทัดแรก พี่ ๆ จะได้ไม่ต้องdingงานกลับไปกลับมาแบบที่ผมเคยเจอ

**17**

หน้าจอที่ compile ผ่าน  
Studio จริง  
(2,830 controls)

**131**

SharePoint columns  
ที่ใช้งานอยู่จริง

**5**

skills  
ลงครั้งเดียว ใช้ได้ทุกโปร  
เจกต์

**0**

โปรแกรมที่ต้องลง  
ไม่ต้องใช้ GitHub

## ถ้าตอนนี้มีเวลา 2 นาที อ่านแค่นี้พอ

### 1 ติดตั้ง — ทำครั้งเดียว ประมาณ 2 นาที

ใน [claude.ai](#) กด **Settings** → **Capabilities** → **Skills** → **Upload skill** แล้วอัปโหลดไฟล์ `.zip` ทั้ง 5 อันที่ผมแนบมาให้ ทีละอัน

Skills เป็นของแต่ละ account และ **ไม่ sync** กัน ทุกคนต้องลงของตัวเอง และตอนผมอัปเดต ทุกคนต้องโหลดใหม่ — ผมจะบอกทุกครั้งว่ารอบไหนต้องโหลด รอบไหนข้ามได้

### 2 ใช้งาน — เปิดแชทใหม่ แล้วพิมพ์สั้น ๆ ว่าจะทำอะไร

ไม่ต้องเรียกชื่อ skill ไม่ต้องอธิบายกฎ Power Apps ไม่ต้องแปะ convention อะไรทั้งนั้น แคบอกงาน เช่น “ทำหน้านี้เป็น Power App ให้หน่อย” แล้วแนบรูปหรือ mockup ไป

กฎ 2,830 controls อยู่ใน skill แล้ว มันจะหยิบมาใช้เอง

### 3 ก่อนปิดแชท — วาง prompt 1 อัน แล้วส่งไฟล์ที่ได้กลับมาให้ผม

prompt อยู่หน้า 9 และอยู่ในไฟล์ `team-quick-feedback.md` ที่แนบมา ใช้เวลาไม่เกิน 2 นาที ไม่ต้องคิดเยอะ ไม่ต้องตัดสินใจอะไร

นี่คือส่วนเดียวที่ผมขอ และเป็นส่วนเดียวที่ทำให้ skill ดีขึ้นเรื่อย ๆ แทนที่จะค่อย ๆ เก่าไป

### ไม่มีอะไรใหม่ที่ต้องเรียนครับ

ยังเป็นการคุยกับ Claude เหมือนเดิมทุกอย่าง ต่างแค่นี้ Claude รู้กฎของเราแล้ว — ไม่ต้องลงโปรแกรม ไม่ต้องใช้ command line และไม่ต้องแตะ **GitHub** เลยสักครั้ง

### ในโฟลเดอร์ที่ผมส่งให้ มีอะไรบ้าง

**5 × .zip** — อัปโหลดเข้า [claude.ai](#) ทีละอัน (ขั้นตอนที่ 1)

**team-quick-feedback.md** — prompt ที่วางท้ายแชท · หน้า 10

**project-style-override.md** — ใช้ตอนโปรเจกต์นี้ใช้ดีไซน์คนละชุด · หน้า 7

**CHANGELOG.md** — ผมทำอะไรไปบ้าง และรอบไหนต้องโหลดใหม่

ไฟล์ PDF นี้

# ทำไมผมถึงทำอันนี้

เริ่มจากสองเรื่องทีพี ๆ บ่นตรงกัน และผมก็เจอเหมือนกันทุกครั้ง:

## ปัญหาข้อ 1

ใช้เวลาขึ้น Power Apps เยอะ กว่าจะเลื่อนแต่ละกล่องได้แต่ละอัน

## ปัญหาข้อ 2

Claude มันยัง generate Power Apps ไม่เก่ง ต้อง prompt เยอะ ๆ ถึงงานกลับไปมานาน กว่าจะได้ YAML code ที่ไม่ error

## สองข้อนี้เป็นเรื่องเดียวกัน

ที่ Claude generate ไม่เก่ง ไม่ใช่เพราะมันเขียน code ไม่เป็น แต่เพราะมันไม่รู้กฎเฉพาะของ **Power Apps** ที่เราต้องเจ็บตัวเองถึงจะรู้ — control type ไหนมีอยู่จริงบ้าง เวอร์ชันต้องเป็นเลขอะไร property เรียงยังไง ค่าแบบไหนทำให้ YAML พังทั้งไฟล์

พอไม่รู้ มันก็เดา และคำตอบที่เดามาจะถูกต้องเสมอ — ชื่อ control ที่ฟังดูมีจริง property ที่ฟังดูสมเหตุสมผล เราถึงไม่รู้ตัวจนกด paste แล้ว error นั่นแหละคือรอบที่กินเวลาเรามากที่สุด

ผมเลยไม่ได้ไปนั่งสอนมันใหม่ทุกแชท แต่ไปวัดกฎจากของจริงที่ **compile** ผ่านแล้ว แล้วใส่ไว้ใน skill ให้มันรู้ตั้งแต่ยังไม่เริ่มพิมพ์

## กฎทุกข้อวัดมาจากของที่ทำงานได้จริง ไม่ได้มาจากเอกสาร

- 17 หน้าจอที่ compile ผ่าน Studio — 2,830 controls, 2,847 property blocks
- 10 SharePoint lists, 131 columns ที่ใช้งานอยู่จริงใน production
- flow package ที่ export ออกมาจาก tenant ของเราเอง

กฎข้อไหนมีตัวเลขแปะอยู่ แปลว่ามันไม่ใช่ความเห็น และตอนที่เอกสาร convention ของเราขัดกับไฟล์ที่ shipped ไปแล้ว — ไฟล์ชนะ เอกสารโดนแก้ (มี 7 ค่าที่เอกสารเราสั่งไว้ แต่ไม่ปรากฏเลยสักครั้งใน 17 หน้าจอ)

# สิ่งที่เปลี่ยนจริง ๆ คือลำดับการทำงาน

ไม่ใช่ที่เราพิมพ์เร็วขึ้น แต่คือตกลงแบบให้จบบนกระดาษก่อน แล้วค่อยสร้าง

ได้ requirement มา



## แชท 1 · planning-powerplatform-solutions

สัมภาษณ์ requirement แล้วออกมาเป็น `solution-spec.yaml` + mockup HTML + PDF ภาพรวมระบบ (UI, database, tech stack)



### \* จุดที่ประหยัดเวลาที่สุดของทั้งกระบวนการ

เอา PDF กับ mockup ไปคุยกับ user ตรงนี้ ก่อนสร้างอะไรทั้งนั้น

เมื่อก่อนเราไปเสียเวลาขึ้น Power Apps ก่อน แล้วค่อยรู้ว่า user อยากได้อีกแบบ ตอนนี้ user เห็นภาพและแก้ตั้งแต่ยังเป็นกระดาษ ซึ่งแค่ 5 นาที ไม่ใช่ครึ่งวัน



## แชท 2

building-sharepoint-lists  
.xlsx สำหรับ copy ลง  
SharePoint

## แชท 3

generating-powerapps-yaml  
.pa.yaml ที่ paste เข้า Studio

## แชท 4

building-powerautomate-flows  
.zip ที่ import ได้



## แชท 5 · writing-app-manuals

คู่มือ admin + คู่มือ user เป็นไฟล์ Word

`solution-spec.yaml` คือของชิ้นเดียวที่เดินทางข้ามแชท

แชท 1 เขียนมันขึ้นมา อีกสี่ตัวอ่านมัน — และแต่ละตัวอ่านเฉพาะส่วนของตัวเอง เอาไฟล์นี้แนบเข้าไปในแชทถัดไป แล้วพิมพ์สั้น ๆ ว่าจะทำอะไร ไม่ต้องเล่าระบบใหม่ทั้งหมดอีกรอบ — อันนี้คือสิ่งที่ประหยัด token ได้มากที่สุด

## 5 skills แต่ละอันแก้ปัญหาอะไร

แต่ละอันสร้างของคนละชนิด ไม่ทับกัน เพราะฉะนั้นพิมพ์งานเข้าไปแล้ว มันจะวิ่งไปเข้าตัวที่ถูกเอง — ไม่ต้องเลือกให้มัน

### 01 planning-powerplatform-solutions

- พูดว่า “ช่วยวางระบบขออนุมัติเปิดบัญชีให้หน่อย” แล้วโยน flowchart / ER diagram / เอกสาร process ที่มีอยู่ให้มัน
- ได้อะไร `solution-spec.yaml` · mockup HTML · PDF ภาพรวมระบบสำหรับเอาไปคุยกับ user
- แก้ปัญหา การที่เราเริ่มขึ้นแอปก่อนตกลงแบบ อันนี้ทำให้ user เห็นและแก้ได้ตั้งแต่ยังไม่ทำอะไรให้แก้
- ต้องรู้ ทำอันนี้ก่อนเสมอ อีกสี่ตัวอ่าน spec ที่มันเขียน · มันจะทำ glossary ก่อนเรื่องอื่นทั้งหมด เพราะของสิ่งเดียวที่ถูกเรียกสองชื่อ จะกลายเป็นสอง list สองหน้าจอ แล้วต้อง migrate ที่หลัง · และมันหยุดสัมภาษณ์เองเมื่อ **spec** ผ่าน **validator** แล้ว ไม่ถามวนไปเรื่อย ๆ

### 02 generating-powerapps-yaml

- พูดว่า “ทำหน้านี้เป็น Power App ให้หน่อย” แล้วแนบ mockup / screenshot / HTML / spec
- ได้อะไร ไฟล์ `.pa.yaml` ที่ paste เข้า Power Apps Studio ได้เลย
- แก้ปัญหา ปัญหาข้อ 2 ของพี ๑ ตรง ๆ — มันรู้ว่ามี control อยู่ 9 ชนิด เวอร์ชันต้องเป็นเลขอะไร property ต้องเรียงยังไง ค่าไหนต้องใส่ block scalar ไม่งั้น YAML พัง และมันรัน **validator** ให้ก่อนส่งงานทุกครั้ง
- ต้องรู้ อันนี้คือตัวที่ผมลงแรงมากที่สุด และเป็นตัวที่ผ่านสนามจริงมาแล้ว — ลองกับโปรเจกต์ที่ไม่เกี่ยวอะไรกับโปรเจกต์ต้นทางเลย ออกมา 191 controls โดยไม่มี control type หรือ property ที่ไม่มีอยู่จริงสักตัว แต่ต้องอ่านหน้า 8 ก่อนเช็ **validator** ครับ

### 03 building-sharepoint-lists

- พูดว่า “สร้าง list ตาม spec นี้ให้หน่อย” (แนบ `solution-spec.yaml` จากเซท 1)
- ได้อะไร ไฟล์ `.xlsx` ที่ทำให้ copy ลง SharePoint ได้ตรง ๆ + สคริปต์สร้าง column
- แก้ปัญหา การมานั่งพิมพ์ column ที่ละอันในเบราว์เซอร์ และกับดัก **delegation** — มันจะไม่ยอมใช้ Lookup column ให้ เพราะ Lookup ไม่ delegate แล้ว gallery จะเจ็บบ ๑ ตัดที่ 500 แถว โดยไม่มี **error** อะไรเลย (131 columns ที่ใช้จริงของเรา ไม่มี Lookup สักอัน — ตั้งใจ)

#### 04 building-powerautomate-flows

- พูดว่า “ทำ flow ส่งเมลตอนมีคนกดส่งคำขอ” — และต้องมีไฟล์ `.zip` ที่ export จาก tenant เราเป็นต้นแบบ
- ได้อะไร ไฟล์ `.zip` ที่ import เข้า Power Automate ได้
- แก้ปัญหา package ที่ import แล้วขึ้น connection ผิดเพี้ยน หรือ error `PackageFlowMissingConnectionMap` — เพราะ legacy package จริง ๆ มี 5 ไฟล์ ไม่ใช่ 2 และสองไฟล์ในนั้น (`apisMap.json`, `connectionsMap.json`) ไม่มีในเอกสารที่ไหนเลยที่เราหาเจอ
- ต้องรู้ **ต้องมี .zip ต้นแบบก่อนเสมอ** ไม่มีแล้วทำไม่ได้จริง ๆ ไม่ใช่มันซับซ้อน — เพราะใน package มีรหัส connection กับ GUID ที่ผูกกับ tenant เรา เราไม่ได้หาไม่ได้ ต้องมาจากไฟล์จริงเท่านั้น สร้างต้นแบบใหม่ใช้เวลา 2 นาที (สร้าง flow เปล่า ๆ 1 action แล้ว export)

#### 05 writing-app-manuals

- พูดว่า “เขียนคู่มือให้หน่อย”
- ได้อะไร คู่มือ admin + คู่มือ user เป็นไฟล์ `.docx` สองอัน จาก spec เดียวกัน
- แก้ปัญหา คู่มือที่ไม่มีใครได้เขียนเพราะไม่มีเวลา และคู่มือสองเล่มที่เขียนแยกกันแล้วเนื้อหาเพี้ยนกันภายในอาทิตย์เดียว
- ต้องรู้ **มันมีโหมด draft ที่ใช้ก่อนสร้างระบบ** — เอาไปให้ user อ่าน แล้ว user บอกว่า “เฮ้ย เราไม่ได้ทำแบบนี้” อันนั้นคือเราเพิ่งจับ requirement ผิดได้ในราคาแค่หนึ่งประโยค ถ้าไปถามเขาว่า “approve spec นี้ได้ไหม” เขาจะพยักหน้าอย่างเดียว

### ไม่ต้องจำชื่อพวกนี้ก็ได้ครับ

แต่ละ skill เขียน description ไว้ให้จับจากวิธีที่คนถามจริง ๆ เช่น “make this a Power App”, “ทำไม gallery ขึ้นแค่ 500 แถว”, “flow import ไม่ได้” — พิมพ์ปัญหาไปตรง ๆ แล้วมันวิ่งไปเข้าตัวที่ถูกต้อง

ถ้าพิมพ์ไปแล้วมันไม่เรียก skill ขึ้นมา นั่นคือบั๊กที่ควรบอกผม ไม่ใช่เรื่องที่พี่ ๆ พิมพ์ผิด

## ใช้อย่างไรให้ token ไม่หมดก่อนงานเสร็จ

พี่ ๆ ใช้ Pro plan เพราะฉะนั้นข้อนี้สำคัญพอ ๆ กับความถูกต้อง

### 1. หนึ่ง stage ต่อหนึ่งแชท

เปิดแชทใหม่ทุกครั้งที่ขึ้น skill ตัวใหม่ ไม่ใช่เรื่องความเป็นระเบียบ แต่เป็นเรื่อง budget ตรง ๆ — ถ้าเอาการสัมภาษณ์ requirement กับการสร้างของสามอย่างมาอยู่แชทเดียวกัน context จะถูกกินไปกับเรื่องที่ stage หลัง ๆ ไม่ได้ใช้ และคุณภาพจะตกก่อนที่ token จะหมดด้วยซ้ำ

### 2. พก `solution-spec.yaml` ข้ามแชท — แแนบเป็นไฟล์ อย่า paste ทั้งก้อน

แนบไฟล์ไป แล้วพิมพ์ว่า “สร้าง SharePoint list ตาม spec นี้” พอไม่ต้องเล่าระบบใหม่ ไม่ต้อง paste YAML ทั้งอันลงในช่องแชท

### 3. prompt สั้น ๆ ก็พอ กฎอยู่ใน skill แล้ว

ไม่ต้องเขียน prompt ยาว ๆ อธิบายว่าอยากได้ YAML แบบไหน ไม่ต้องแปะ convention ไม่ต้องเตือนมันเรื่อง property ordering — ทุกอย่างนั้นอยู่ใน skill แล้ว การเขียนซ้ำคือการจ่าย token ซ้ำ

### 4. มีอะไรจะแก้ บอกให้ครบในรอบเดียว

สะสมสิ่งที่จะแก้ให้ครบแล้วส่งไปที่เดียว ประหยัดกว่าส่งทีละข้อสามรอบมาก และผลลัพธ์มักจะดีกว่าด้วย เพราะมันเห็นภาพรวมของสิ่งที่เราต้องการ

ถ้า **validator** เตือนขึ้นมาเป็นสิบบรรทัดเพราะโปรเจกต์นี้ใช้สีกับขนาดคนละชุด — อย่าไปแก้อะไร และอย่าเริ่มชินกับการมองข้าม warning ให้เปิดไฟล์ `project-style-override.md` ที่แนบมา แล้ววาง prompt ในนั้นตอนเริ่มแชท มันจะตั้งค่าสไตล์ของโปรเจกต์นี้ให้ โดยไม่ต้องแตะ skill เลย

## “ผ่าน validator” แปลว่า paste ได้ ไม่ได้แปลว่าหน้าจอลูก

ทุก skill จะรัน validator ให้ก่อนส่งงาน และมันจับได้เยอะจริง ๆ — control type ที่ไม่มีอยู่จริง, property ที่เรียงผิด, ค่าที่ทำให้ YAML พัง, ขนาดที่หลุดจากชุดที่วัดมา

แต่มันอ่านแค่ตัวหนังสือ มันไม่ได้คำนวณ **layout** มันไม่มีทางรู้ว่ากล่องนี้เตี้ยเกินกว่าของที่อยู่ข้างใน หรือเส้นขอบของลูกโดนขอบของพ่อบังอยู่

### เรื่องนี้ผมพลาดเอง และมันทำให้เสียเวลาไป 3 รอบ

ตอนแรกผมให้ validator ขึ้นข้อความว่า “Clean. Safe to paste into Power Apps Studio.” ซึ่งมันเคลมเกินจริง เพราะสิ่งที่มันพิสูจน์ได้จริง ๆ มีแค่ “ไฟล์นี้ parse ผ่าน”

ผลคือผมรัน validator ผ่าน แล้วก็เชื่อมั่น — สามรอบติด ทั้งที่หน้าจอออกมาเพี้ยนทุกรอบ ตอนนั้นแก้แล้ว มันจะขึ้นว่า “No errors found. This should PARSE and paste.” แล้วตามด้วยรายการที่มันไม่ได้ตรวจ ทุกครั้ง

#### ✓ มันตรวจให้

- control type และเวอร์ชันมีจริงไหม
- property เรียงถูกไหม
- ค่าที่ต้องใส่ block scalar
- X / Y ที่ไม่ควรมี
- property ที่หน้าจอต้งมี
- enum member ที่ยังไม่มีใครยืนยัน

#### ✗ มันตรวจไม่ได้

- กล่องสูงพอใส่ของข้างในไหม
- เส้นขอบโดนพ่อบังหรือเปล่า
- `=Parent.Width` ล้นพอที่มี padding
- อะไรก็ตามที่ขึ้นกับข้อมูลจริงตอน run

### เพราะฉะนั้น: paste เข้าไป ดูหน้าจอ แล้วบอกมันว่าเห็นอะไร

การบอกว่า “กล่องนี้เตี้ยไป ตัวหนังสือล้น” หนึ่งรอบ เร็วกว่านั่งเดากับมันสามรอบเยอะมาก — และรอบนั้นคือรอบที่ตาพี่ ๆ ทำได้ดีกว่ามันแน่นอน



# ทำให้มันเก่งขึ้นเรื่อย ๆ ได้ยังไง โดยไม่ต้องแตะ GitHub

ของพวกนี้สร้างมาจากระบบเดียว ทุกโปรเจกต์หลังจากนั้นจะเจอเรื่องที่ 17 หน้าจอนั้นไม่เคยมี ถ้าไม่มีทาง  
ส่งกลับ ทุกคนก็จะไปนั่งค้นพบการแก้เดิม ๆ ซ้ำกันเจียบ ๆ คนละมุม แล้ว skill ก็จะไม่ดีขึ้นเลย  
เลยมีทางส่งกลับอยู่ 3 ทาง และทางที่ผมอยากให้ใช้มากที่สุดคือทางแรก ซึ่งใช้เวลา 2 นาที

## ทาง 1 · ทุกคน

2 นาที  
ทุกครั้งทำงานจริง  
เสร็จ

### วาง prompt ท้ายแชท แล้วส่งไฟล์มาให้ผม

prompt เต็ม ๆ อยู่หน้าถัดไป และในไฟล์ `team-quick-feedback.md` Claude จะเขียน  
บันทึกสั้น ๆ ออกมาว่าอะไรต้องแก้บ้าง เซฟแล้วส่งมาทาง Teams จบ

ต้องท้ายแชทเท่านั้น แชทยาว ๆ จะถูกย่อความ แล้วสิ่งที่เหลือรอดคือสรุปที่ฟังดูสมเหตุสมผล  
มากกว่าที่ถูกต้อง ถามวันศุกร์ถึงงานวันอังคาร จะได้เรื่องแต่งที่มีผิวสัมผัสของข้อเท็จจริง

และอย่ากรอกก่อนส่ง — อย่าคิดแทนว่าเรื่องนี้สำคัญพอไหม มีคนบอกไปหรือยัง หรือเป็นความผิดพลาด  
เองหรือเปล่า ส่งมาเถอะครับ ตัดสินใจเป็นงานผม และผมทำได้ดีกว่ามากตอนมีห้ารายงาน มากกว่า  
มีอันเดียว

## ทาง 2 · ทำเองได้

ไม่ต้องรอผม  
เฉพาะเรื่องสไลด์

### โปรเจกต์นี้ใช้ดีไซน์คนละชุด — ตั้งค่าเองได้ในแชทนั่นเลย

สี ขนาดฟอนต์ ระยะห่าง prefix ตัวแปร — พวกนี้เป็นธรรมเนียมของทีม ไม่ใช่กฎของ  
แพลตฟอร์ม เปลี่ยนได้โดยไม่ต้องรอใคร ใช้ prompt ในไฟล์ `project-style-  
override.md`

อยากรู้ว่าที่เดือนมามีอันไหนฟังจริงบ้าง พิมพ์ว่า “validate with --platform-only แล้วเทียบสองรอบ  
ใหญ่” อะไรที่หายไปคือเรื่องสไลด์ อะไรที่ยังอยู่คือเรื่องที่ Studio จะปฏิเสธจริง

## ทาง 3 · ผมทำ

ประมาณเดือนละครั้ง

### ผมรวมทุกอย่างที่ส่งมา แกะ skill แล้วส่ง zip ใหม่ให้

พร้อมบอกทุกครั้งว่ารอบนี้ต้องโหลดใหม่ไหม หรือข้ามได้ หน้าที่พี่ ๆ มีแค่โหลดกับอัป  
โหลดทับ

และผมจะบอกด้วยว่าเรื่องไหนผมไม่ได้แก้ และเพราะอะไร — คนที่ส่งมาสองครั้งแล้วไม่เห็นอะไร  
เกิดขึ้น จะเลิกส่ง แล้วผมก็เสียข้อมูลนั้นไปถาวร

เส้นแบ่งง่าย ๆ ว่าจะใช้ทางไหน: ถ้า **Studio** หรือ **SharePoint** ปฏิเสธจริง ๆ → ทาง 1 ส่งมาให้ผม เพราะทุกคน  
ต้องได้กฏนั้น · ถ้ามันทำงานได้ แค่ไม่ตรงสไลด์โปรเจกต์นี้ → ทาง 2 แก่เองได้เลย

## วางอันนี้ก่อนปิดแชท

ถ้าก๊อปปี้จากไฟล์ `team-quick-feedback.md` ที่แนบมาได้ ก็ก๊อปปี้จากไฟล์ดีกว่าครับ จะได้ไม่มีอักขระหลุดตอนคัดลอกจาก PDF

We are done. Before I close this chat, write a short feedback record so the skill that guided this work can be improved. Be blunt – this is for fixing the skill, not for reassuring me. An honest account of what you got wrong is the useful thing here.

Output one YAML code block and nothing else. No summary paragraph, and do not re-print the artifact – I already have it.

### meta:

```
skill: <which skill guided this>
skill_version: <the version comment in its SKILL.md, if you can see it>
what_we_built: <one line>
rounds: <how many times I sent it back for corrections>
first_attempt_validated: <true|false – did your FIRST output pass the validator?>
```

# Exact text the platform showed – Power Apps Studio, SharePoint, or the flow importer.  
# Copy it character for character, including the Location. Do not paraphrase. This is the  
# most valuable field in the file: every verbatim error so far has become a static check.  
**studio\_errors\_verbatim:** []

### fixes:

```
- element: <the exact control, column, or action. "the layout was wrong" is unusable>
  wrong: <what you produced>
  right: <what it had to become>
  caught_by: validator | studio | me-eyeballing | you-self-corrected
  tier: platform | house | unclear
  rule_status: absent | present-but-wrong | present-and-ignored
  rule_quote: <if present-but-wrong or present-and-ignored, quote the SKILL.md line>
  times: <rounds this same thing took>
```

### biggest\_miss: |

<Free text, and answer it even if fixes is empty. What would have saved the most round trips if the skill had said it on page one? If something felt structurally wrong rather than individually wrong, this is where it goes.>

tier: `platform` means Studio or SharePoint REFUSED it – universal, everyone needs the fix.  
`house` means it worked and just did not match our conventions. Unsure? Write `unclear`.

rule\_status decides what the fix is. `absent` → add a rule. `present-but-wrong` → correct it. `present-and-ignored` → the rule was there, it was correct, and you did not follow it. Say so. It is the most valuable line you can write and the one you will most want to avoid. It means the rule is buried, or worded as advice where an instruction was needed.

Quote any value containing a colon-space, because this record is YAML too:  
wrong: "Control: Image@2.2.0"

If a section is genuinely empty, return it empty. `fixes: []` is a real answer.

แล้วเซฟไฟล์ที่ได้ ส่งมาทาง Teams — จบครับ ไม่ต้องแก้ ไม่ต้องจัดรูปแบบ ไม่ต้องคิดกรอง

## คำถามที่น่าจะจะมี

### ต้องลง GitHub ไหม / ต้องลงโปรแกรมอะไรไหม

ไม่ต้องเลยครับ มีแค่ claude.ai กับเบราวเซอร์ไฟล์ zip ผมส่งให้ทาง Teams และ feedback ก็ส่งกลับมาทาง Teams เหมือนกัน

### ผมลง skill แล้ว เพื่อนในทีมได้ด้วยไหม

ไม่ได้ครับ Skills บน claude.ai เป็นของแต่ละ account และไม่ sync กัน ทุกคนต้องอัปเดตเอง — และตอนผมออกเวอร์ชันใหม่ ทุกคนต้องโหลดใหม่เอง ผมจะบอกทุกครั้งว่ารอบไหนควรโหลด

### ต้องลงครบทั้ง 5 อันเลยไหม

ลงครบดีที่สุดครับ เพราะมันถูกเขียนให้ส่งงานต่อกัน แต่ถ้าจะเริ่มอันเดียวก่อน เอา `generating-powerapps-yaml` — เป็นตัวที่กินเวลาให้เร็วที่สุด

### มันจะกิน token เยอะไหม

น้อยกว่าเดิมครับ เพราะถูกอยู่ใน skill แล้ว เราไม่ต้องพิมพ์อธิบายซ้ำทุกแชท และไม่ต้องdingานกลับไปมาหลายรอบ — รอบที่ติดกลับนั้นแหละที่แพงที่สุด ขอแค่ทำตามกติกาหน้า 7: หนึ่ง stage ต่อหนึ่งแชท

### ถ้าผมทำมันพัง / แก้อะไรผิดไป

พังยากครับ และถ้าพังจริงก็แค่โหลด zip ใหม่มาอัปเดตทับ ของเดิมอยู่กับผมครบ ที่ไม่อยากให้ทำมีอย่างเดียวคือ อย่าไปแก้ไฟล์ `validate_*.py` และอย่าปล่อยให้ตัวเองชินกับการมองข้าม warning — วันที่ warning กลายเป็นสิ่งที่ทุกคนข้าม คือวันที่มันหมดประโยชน์

### พิมพ์ไปแล้วมันไม่เรียก skill ขึ้นมาเลย

บอกผมได้เลยครับ นั่นคือ description ของ skill เขียนไม่ครอบคลุมวิธีที่คนถามจริง เป็นบั๊กที่ผมแก้ได้ ไม่ใช่เรื่องที่พีๆ ต้องพิมพ์ให้ถูกใจมัน

### มันเคยผ่านสนามจริงหรือยัง

ผ่านแล้วหนึ่งโปรเจกต์ ที่ไม่เกี่ยวอะไรกับโปรเจกต์ด้านทางเลย ออกมา 191 controls ไม่มี control type หรือ property ที่ไม่มีอยู่จริงสักตัว และไม่เตือนเรื่องสืเลยแม้จะใช้ชุดสืคนละชุดโดยสิ้นเชิง — ซึ่งเป็นสิ่งที่ตั้งใจให้เป็นส่วนที่พลาดก็มี และมันกลายเป็นเรื่องในหน้า 8

### สิ่งเดียวที่ผมขอ

วาง prompt หน้า 10 ท้ายแชท แล้วส่งไฟล์กลับมา

ของพวกนี้จะดีขึ้นตามจำนวนคนที่ส่งกลับ ไม่ใช่ตามเวลาที่ผมใส่ลงไป คนที่ส่งมาห้าครั้ง กำลังเขียนเครื่องมือให้ตัวเองใช้ในอีกหกเดือนข้างหน้า

ติดตรงไหน ทักมาได้เลยครับ ไม่ต้องเกรงใจ — N'Vin